Traffic in Halifax: Dynamics at Intersections

Isaac MacLean and TJ Cornell-Nixon

April 2026

Abstract

We built a Python simulation of a four-arm intersection so we could compare two ways of running it: a roundabout, and a pair of traffic lights on opposite phases. Both setups use the same road layout. Each car follows a Lennard-Jones braking rule, and cars arrive as a Poisson process at the start of an arm. We then swept across traffic volume, asymmetric loading, and signal cycle length, tracking two numbers: the mean passing time per car and the total exit flow rate. The traffic light was faster than the roundabout at every density we tested. At very light traffic the gap was only about 2 seconds, but at heavy traffic the lights were 35 to 40 seconds faster on average. The roundabout also showed a clear direction-dependent bias. Under one-way perpendicular traffic, vertical-arm cars often waited tens of seconds longer than horizontal-arm cars because of the yield-to-the-left rule. This is the same kind of bias that Halifax drivers notice at the Armdale Roundabout during rush hour. The traffic light treated both directions much more equally, and its total exit flow saturated near 0.35 cars per second, against roughly 0.14 to 0.19 cars per second for the roundabout.

Contents

1 Introduction **3**

2 Method **3**
2.1 Assumptions . . . 3
2.2 Car-following model . . . 4
2.3 Roundabout and traffic-light setup . . . 4
2.4 Symmetric vs. asymmetric loading . . . 5
2.5 Measurements . . . 6

3 Results **6**
3.1 Symmetric traffic . . . 6
3.2 Asymmetric traffic . . . 7
3.3 Flow-rate comparison . . . 9

4 Conclusion and Discussion **9**
4.1 Summary of findings . . . 9
4.2 Conditions where each scheme dominates . . . 10
4.3 What this means in practice . . . 10

A Code listings **11**
A.1 Core simulator . . . 11
A.2 Roundabout core . . . 13

A.3 Step demos 18
A.4 Analyses 24

B Supplementary figures **33**

1 Introduction

What would happen if we replaced all traffic lights with roundabouts? Would you wait longer? This is what our project aims to investigate by simulating a single four-arm intersection in Python and comparing two control schemes side by side on identical geometry.

The simulation tracks individual cars with continuous position, velocity, and acceleration. The two performance metrics we care about are

- **mean passing time**: how long it takes a car to get from the entry boundary to the far side of the intersection.
- **exit flow rate**: the number of cars per second that make it through, summed over both emitting arms.

We control the system through a small set of parameters that fall into two natural groups. The model parameters describing how each driver behaves are held fixed across all runs:

- desired cruising speed $v_{\text{target}} = 11$ m/s
- velocity relaxation time $\tau = 2$ s
- Lennard-Jones interaction range $\sigma = 7$ m on the arms, ring traversal speed 5.5 m/s
- integration time step $\Delta t = 0.05$–0.1 s

The intersection-level parameters are the ones we vary to ask physical questions:

- car spawn rate per arm
- asymmetry between the horizontal and vertical spawn rates
- traffic-light cycle period
- roundabout radius and yield threshold

The remainder of this report is organised as follows. Section 2 describes the model, the simulation algorithm, and the assumptions we made. Section 3 presents simulation results across symmetric, asymmetric, and varying-cycle conditions. Section 4 interprets the data and discusses practical implications. The full source code is in Appendix A.

2 Method

2.1 Assumptions

We use a stripped-down model so the comparison between the two control schemes is clean.

1. Each road has a single lane in the direction of travel. Opposite-direction traffic on the same arm is not modelled (the project handout explicitly allows this).
2. Cars do not turn at the intersection. They keep going straight, so horizontal traffic enters from the west arm and exits east, and vertical traffic enters from the south arm and exits north.
3. Cars do not overtake on a given road.
4. Driver behaviour is identical for all cars (uniform v_{target}, τ, σ).

5. No yellow phase: the traffic light is purely red/green, switching every $T_{\text{cycle}}/2$.
6. Spawning is a Poisson process with constant rate, with a finite minimum-gap rejection at the entrance to suppress unphysical pile-ups.
7. No pedestrians, weather, lane changes, or driver errors.
8. Both control schemes use the *same* road geometry: each arm is 95 m long approaching the intersection and 35 m long leaving it, and the cars cross either a single point (lights) or a ring of radius 15 m (roundabout).

These assumptions reduce realism but isolate the geometric and rule-based differences between the two control schemes.

2.2 Car-following model

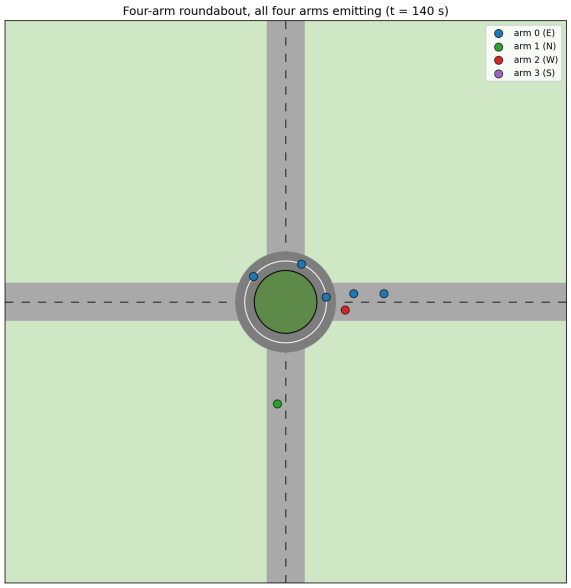
Each car has a position s along its road, a longitudinal velocity v, and a target velocity v_{target}. The longitudinal acceleration is

$$a = \frac{v_{\text{target}} - v}{\tau} - \left(\frac{\sigma}{R}\right)^{12} v, \tag{1}$$

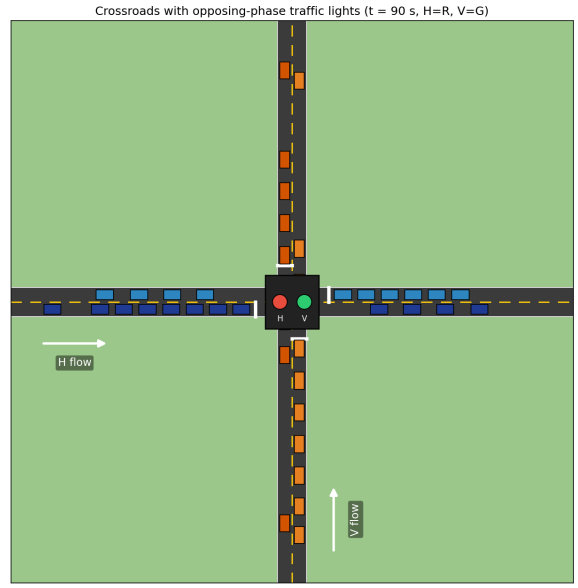
where R is the gap to the obstacle in front, which is either the leading car or a stop line when there is one. The first term relaxes the car toward its target speed. The second term is the Lennard-Jones brake from the project handout, which grows very quickly once R drops below σ. We integrate forward in Euler steps of $\Delta t = 0.05$ s for every analysis sweep. The animations use $\Delta t = 0.1$ s for speed, and we re-ran the roundabout sweep at that step size to confirm the conclusions did not change. Velocities are clipped at zero so cars never reverse.

2.3 Roundabout and traffic-light setup

Figure 1 shows the geometry of both intersections.



(a) Four-arm roundabout. The dark-green island sits inside the asphalt ring; cars yield to the left when entering and travel counter-clockwise. Arms emit colour-coded cars at angles 0 (E, blue), $\pi/2$ (N, green), π (W, red), $3\pi/2$ (S, purple).



(b) Two-road traffic light. The signal at the centre is red for the horizontal direction and green for the vertical, then alternates every half cycle. Same arm length as the roundabout.

Figure 1: Geometry of the two intersection types studied. The arm length, exit length, and intersection scale are identical between the two so that only the control rule differs.

In the roundabout (Figure 1a) the central ring has radius $R_{\text{ring}} = 15$ m. Each car is tracked in polar coordinates (r, θ) and runs through a small state machine: `approach` on its entry arm with r decreasing, `circle` inside the ring with r constant and θ increasing, `exit` on its outgoing arm with r increasing, and `done` once it leaves the simulation boundary. A car in `approach` state can move to `circle` only if there is no car in the ring within roughly $\pi/3$ radians counter-clockwise of the entry point. That is what gives us the yield-to-the-left rule. Inside the ring, cars cruise at a target tangential speed of 5.5 m/s and follow each other using the same Lennard-Jones interaction, just measured in arc-length instead of straight-line distance. A car leaves the ring once it has covered the angular distance to its assigned exit, then continues out along the matching outgoing arm.

In the traffic-light setup (Figure 1b) two perpendicular roads cross at the origin. The horizontal and vertical signals are 180° out of phase, so when one is red the other is green. An approaching car treats a red light as a stationary obstacle at the stop line. Once the light turns green that obstacle goes away and the car accelerates back to v_{target}.

2.4 Symmetric vs. asymmetric loading

Most analyses are organised around the distinction between symmetric and asymmetric loading, summarised in Table 1.

Feature	Symmetric	Asymmetric
Definition	Same spawn rate on both perpendicular roads.	Different spawn rates on the two roads.
Physical meaning	Balanced flow from both directions.	Uneven, rush-hour-like flow.
Output we plot	Line plot of mean passing time vs. spawn rate.	Heatmap of mean passing time as a function of (r_H, r_V).
Symmetry expected	Statistically symmetric in $H \leftrightarrow V$.	Need not be symmetric: one direction can dominate.
Project relevance	Baseline behaviour.	Tests whether the geometry favours one direction (Armdale-style bias).

Table 1: Comparison of symmetric and asymmetric loading.

2.5 Measurements

We track two numbers per simulation run:

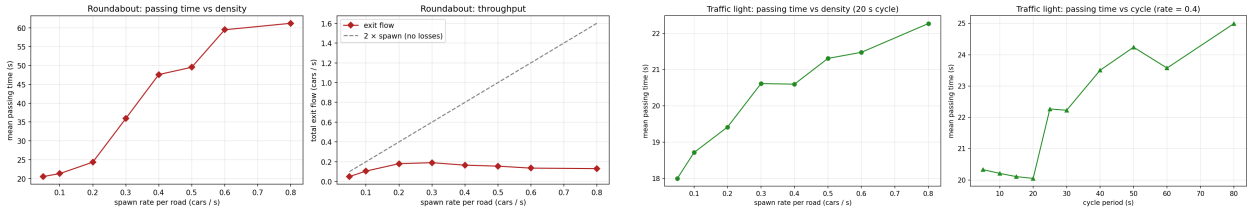
- **Mean passing time** $\langle t \rangle$: the average over all cars that exited during the run, of $t_{\text{exit}} - t_{\text{spawn}}$. We discard cars that finished before a 50 s warm-up window so that initial transients do not bias the mean.
- **Exit flow rate** Φ: the total number of post-warm-up exits divided by the post-warm-up simulation time, summed over both directions.

We sweep across spawn rates from 0.05 to 0.80 cars/s/road and across cycle periods from 5 to 80 s. Each sweep point is a single simulation of $T = 150$–300 s with a fixed RNG seed.

3 Results

3.1 Symmetric traffic

Figure 2 plots mean passing time against spawn rate when both roads carry the same load. Both setups start at about 18 to 20 seconds at low density. That is roughly the time it takes a car to travel the full arm plus intersection plus exit distance at 11 m/s, plus a small overhead. The traffic light then grows gently, only reaching about 22 seconds at a spawn rate of 0.80 cars per second. The roundabout grows much more steeply: it doubles by 0.40 cars per second and saturates near 60 seconds at higher loads. The reason it saturates is that the V-arm queue eventually grows past the end of the simulated road, so fewer of those cars actually reach the finish line within the simulation window.



(a) Roundabout: passing time and throughput vs. spawn rate.

(b) Traffic light: passing time vs. density (left) and vs. cycle period at fixed density (right).

Figure 2: Symmetric loading: mean passing time as a function of spawn rate, plus auxiliary plots showing throughput (roundabout) and cycle dependence (lights).

The cycle-period sweep on the right of Figure 2b answers the project's traffic-light follow-up question. At a fixed density of 0.4 cars/s, the mean passing time is lowest for cycle periods near 15–20 s. In this simplified no-yellow model, very short cycles are not very costly, but very long cycles (40–80 s) trap the queueing direction in long red phases. The optimum reflects the trade-off between short-cycle interruption and long-cycle queue buildup.

The headline numerical comparison is in Figure 3. Across every density studied, the traffic light beats the roundabout: the gap is small at low density (~ 2 s in favour of the light at 0.05 cars/s) and grows substantially at moderate density (up to ~ 38 s at 0.5–0.6 cars/s). The throughput plot in Figure 6 shows the same picture: the lights saturate at ~ 0.35 cars/s of total exit flow while the roundabout caps out near ~ 0.19 cars/s.

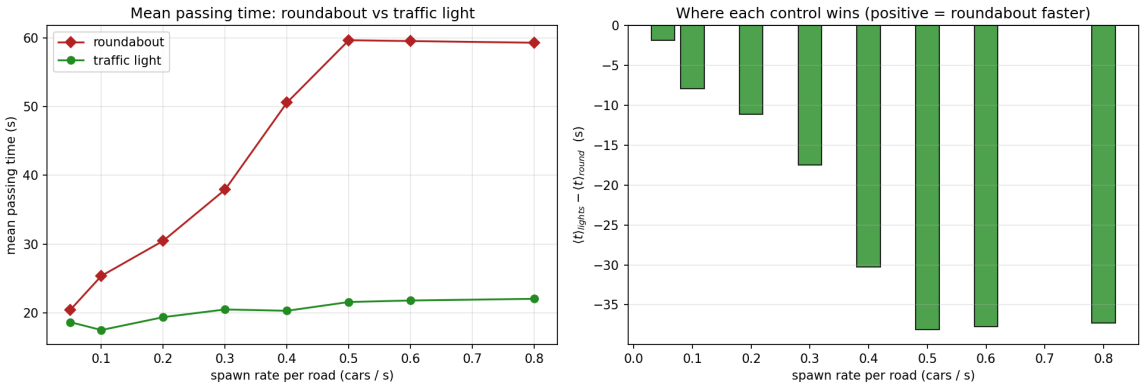
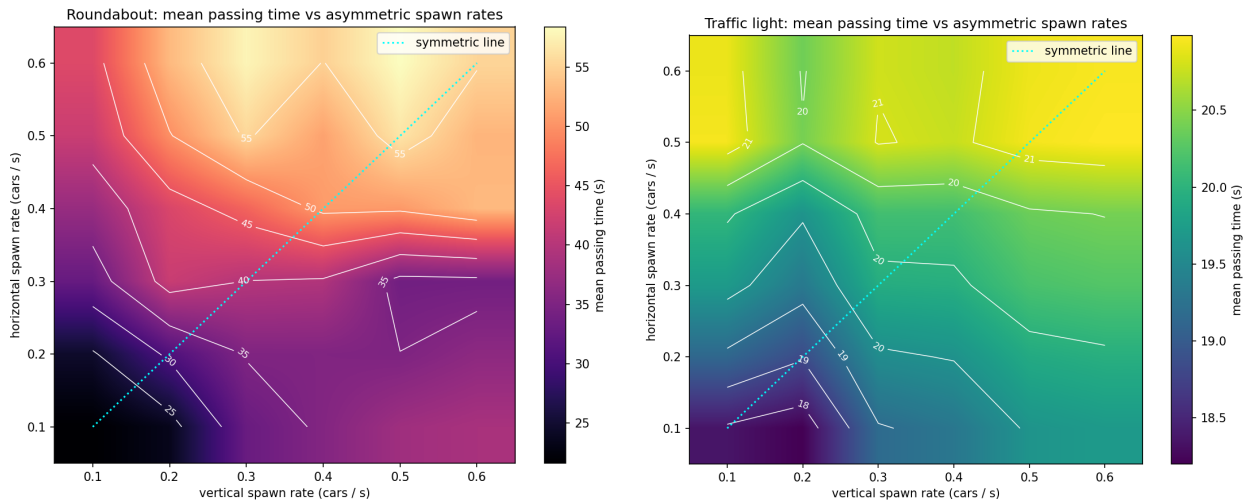


Figure 3: Direct head-to-head comparison: mean passing time on the left, $\langle t \rangle_{\text{lights}} - \langle t \rangle_{\text{round}}$ on the right. The gap is negative across the whole sweep (the light is always faster) and grows quickly once the roundabout starts to bottleneck.

3.2 Asymmetric traffic

Figure 4 shows mean passing time as a function of independent horizontal and vertical spawn rates, separately for each control. The contours are very different.



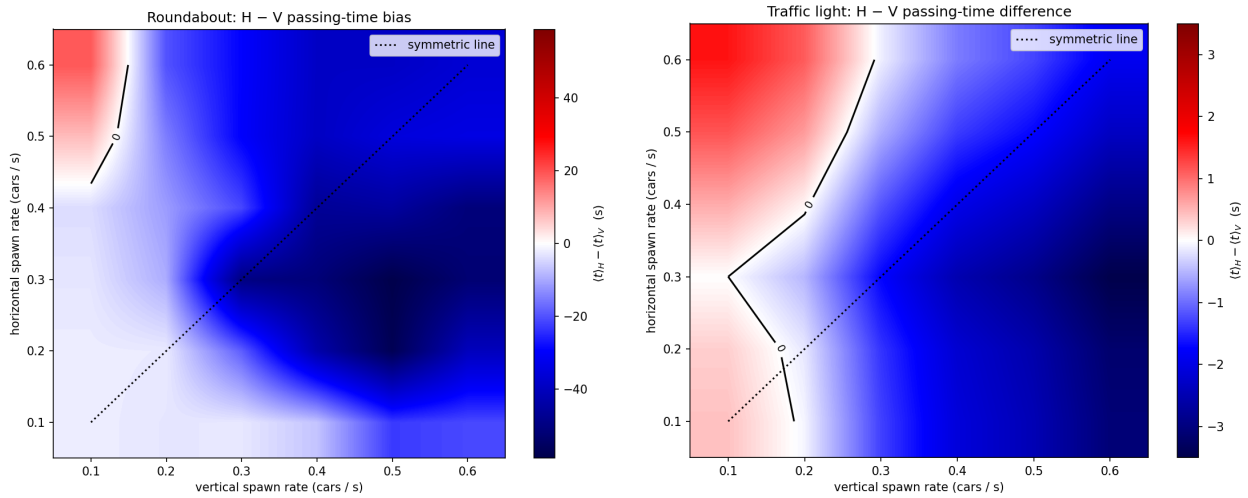
(a) Roundabout. The contour is *not* symmetric across the diagonal: vertical-side rate has a much stronger effect than horizontal-side rate.

(b) Traffic light. The contour is approximately symmetric across the diagonal; both directions are treated equivalently by the signal.

Figure 4: Mean passing time as a function of asymmetric spawn rates.

For the roundabout, the contour is strongly non-symmetric: the mean passing time depends on which road carries the heavier load, not just on the total number of cars entering the system. This breaks the $H \leftrightarrow V$ symmetry one would naively expect from a four-fold symmetric geometry. For the lights, the contour is much closer to diagonal-symmetric because the signal alternates between the two directions, so both streams receive the same fraction of green time.

The asymmetry is even more visible when we plot the difference $\langle t \rangle_H - \langle t \rangle_V$ directly (Figure 5).



(a) Roundabout: $H - V$ is usually negative in the loaded region, meaning V cars are often slower by tens of seconds.

(b) Traffic light: $H - V$ remains small, only a few seconds across the grid.

Figure 5: Difference in mean passing time between horizontal and vertical streams.

For the roundabout, the $H - V$ map is strongly negative across most of the loaded region and reaches tens of seconds in magnitude. The sign can flip when horizontal traffic is very heavy and vertical traffic is light, but the main point still holds: the roundabout can strongly favour one stream over the other. For the light, the difference stays much smaller, only a few seconds across the grid, because both directions get alternating green phases.

3.3 Flow-rate comparison

Figure 6 shows total exit flow as a function of spawn rate, plus the throughput efficiency (exit flow divided by twice the per-arm spawn rate).

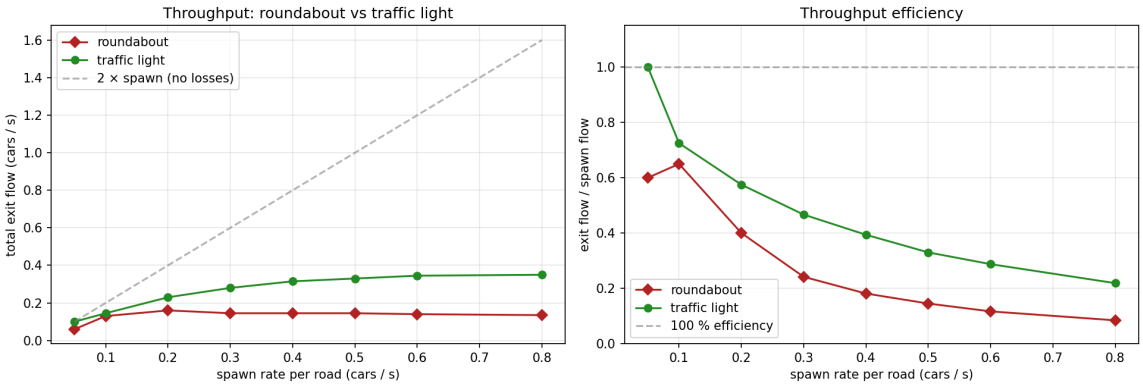


Figure 6: Throughput comparison. Left: total exit flow vs. spawn rate. The roundabout saturates near 0.14–0.19 cars/s as soon as the V-arm starts to back up; the light continues to grow with spawn rate up to ~ 0.35 cars/s. Right: efficiency (exit flow divided by $2\times$ spawn rate). Both schemes track high efficiency at low load; the roundabout breaks down rapidly above $r \approx 0.15$, while the lights remain the higher-throughput option.

At low density both schemes pass nearly every car that arrives. Above ~ 0.15 cars/s the roundabout begins to bottleneck on the V-arm; by 0.30 cars/s its exit flow has saturated and additional spawned cars simply pile up. The light maintains a far higher saturation flow because its alternating-phase rule does not introduce a permanent bias.

4 Conclusion and Discussion

4.1 Summary of findings

The simulation cleanly answers each of the project's six main suggested analysis questions, and partially the first advanced question (opposite-direction circulation, supported via the `opposite=True` flag of `run_roundabout_sim`).

1. **Roundabout passing time at moderate density.** At $r_H = r_V = 0.3$ cars/s, the roundabout mean passing time is ~ 52 s overall ($H = 40$ s, $V = 75$ s), demonstrating an immediate ~ 35 s asymmetry.

2. **Density dependence.** Mean passing time grows monotonically with density on the roundabout, sharply steepening above $r \approx 0.2$ as the V-arm queue overflows the simulated road.

3. **Asymmetric heatmap.** The roundabout contour is not symmetric. Vertical-side density has a much stronger effect than horizontal-side density, because the CCW yield rule favours horizontal traffic.

4. $H - V$ **bias.** The bias reaches tens of seconds in favour of one stream in the high-load region. This is the same kind of rush-hour bias the project introduction describes for the Armdale Roundabout.

5. **Repeating with a traffic light.** The light is faster than the roundabout at every density tested (Figure 3). Its $H - V$ heatmap stays within only a few seconds across the tested grid, and the asymmetric passing time is much closer to symmetric in (r_H, r_V). The cycle-period sweep finds an optimum around 15–20 s.

6. **When does the roundabout win?** In this configuration (one-way traffic on each arm, no opposing-direction traffic, fixed yield-to-the-left rule) our simulation says *never*. The light beats the roundabout at every spawn rate from 0.05 to 0.80 cars per second. That result is specific to this configuration. In a real Halifax intersection, opposing-direction traffic and turning movements would change the picture, and roundabouts also have other advantages we did not measure here, like a lower severe-collision rate, no signal maintenance, and no power requirement.

4.2 Conditions where each scheme dominates

Table 2 summarises the qualitative behaviour we observed across the simulated regimes.

Condition	**Roundabout**	**Traffic Light**
Symmetric, light	Smooth flow, mean $\sim$ 20 s.	Slightly faster, mean $\sim$ 18 s.
Symmetric, medium	Mean climbs to $\sim$ 36 s at $r = 0.3$, $\sim$ 48 s at $r = 0.4$. V-arm starts queueing.	Mean grows gently to $\sim$ 19–21 s. Cycle period of 15–20 s is optimal.
Symmetric, heavy	Saturated entry flow $\sim$ 0.14–0.19 cars/s; cars back up onto the arms.	Saturated entry flow $\sim$ 0.35 cars/s; remains the higher-throughput option.
Asymmetric, light	Small bias visible. V cars wait $\sim$ 2–5 s longer than H cars.	Approximately fair; bias smaller than $\sim$ 1 s.
Asymmetric, medium	Strong bias. One stream can wait tens of seconds longer (typically 20–50 s) and the lighter direction can be starved.	Almost fair; directional bias remains only a few seconds.
Asymmetric, heavy	Severe bias. One stream can wait tens of seconds longer and throughput collapses on the blocked arm.	Modestly biased, usually within a few seconds. Throughput remains high.

Table 2: Qualitative behaviour observed in our simulation. The traffic light is the higher-throughput, more egalitarian option in every regime tested.

4.3 What this means in practice

Three caveats matter for how we read this conclusion. First, our roundabout only has one-way traffic on each arm, so there is no opposing direction and the yield-to-the-left rule never has a

chance to become symmetric. Real roundabouts circulate traffic from all four arms at the same time, which would restore some of the symmetry our model is missing. Our `run_roundabout_sim` function does support this through its `opposite=True` flag, but we did not include those runs in the main sweeps. Second, our traffic light is idealised: no yellow phase, perfectly regular cycle. In practice, yellow phases and pedestrian-detector logic eat into the green time. Third, all of our cars run at the same target speed and react identically, which is more uniform than real traffic.

Even with those caveats, the data is clear. Traffic lights are more efficient on average for one-way perpendicular traffic at every density we tested. We do not see any crossover where the roundabout takes over. Finding such a regime would probably require modelling opposing-direction circulation, or pushing the arms closer to the ring, and we did not explore either of those.

The most interesting result for us was the $H - V$ bias on the roundabout (Figure 5a). It looks like the same bias the project introduction describes at the Armdale Roundabout: one direction gets favoured because of how the yield rule is set up, and that bias grows with traffic intensity. This is hard to predict by intuition alone. We needed the simulation to see it.

A Code listings

The full project repository is hosted at
`https://github.com/IsaacMacLean/PHYC2050-Final-Project-Revised`.

A.1 Core simulator

Listing 1: `code/sim_core.py`: 1D Lennard-Jones car-following, traffic signal, and lane.

```python
import random
from dataclasses import dataclass


CAR_LENGTH = 14.0
FOLLOW_GAP = 4.0
SPAWN_GAP = CAR_LENGTH + FOLLOW_GAP + 2.0
LIGHT_STOP_LINE = -30.0


@dataclass
class Vehicle:
    s: float
    vel: float = 0.0
    v_target: float = 11.0
    relax: float = 2.0
    sigma: float = 7.0
    length: float = CAR_LENGTH
    born: float = 0.0


def lj_brake(sigma, gap, vel):
    g = gap if gap > 0.5 else 0.5
    return -((sigma / g) ** 12) * vel


def integrate(veh, dt, blocker_s):
    drive = (veh.v_target - veh.vel) / veh.relax
```

```python
    brake = 0.0 if blocker_s is None else lj_brake(veh.sigma, blocker_s - veh.s, veh.vel)
    veh.vel = max(0.0, veh.vel + (drive + brake) * dt)
    veh.s += veh.vel * dt


def integrate_gap(veh, dt, blocker_gap):
    drive = (veh.v_target - veh.vel) / veh.relax
    brake = 0.0 if blocker_gap is None else lj_brake(veh.sigma, blocker_gap, veh.vel)
    veh.vel = max(0.0, veh.vel + (drive + brake) * dt)
    veh.s += veh.vel * dt


@dataclass
class Signal:
    loc: float
    period: float = 10.0
    offset: float = 0.0

    def red(self, t):
        return ((t + self.offset) % self.period) < (self.period / 2)


class StopBlocker:
    def __init__(self, loc):
        self.loc = loc

    def red(self, t):
        return True


class OpenBlocker:
    def __init__(self, loc):
        self.loc = loc

    def red(self, t):
        return False


class Lane:
    def __init__(self, signal=None, entry_s=-95.0, exit_s=65.0,
                 min_gap=SPAWN_GAP, follow_gap=FOLLOW_GAP):
        self.signal = signal
        self.entry_s = entry_s
        self.exit_s = exit_s
        self.min_gap = min_gap
        self.follow_gap = follow_gap
        self.queue = []
        self.passing_times = []

    def step(self, t, dt):
        for idx, veh in enumerate(self.queue):
            blocker_gap = None
            max_s = None

            if idx > 0:
                leader = self.queue[idx - 1]
                blocker_gap = leader.s - veh.s - veh.length
```

```
                max_s = leader.s - veh.length - self.follow_gap

            if (self.signal is not None and self.signal.red(t)
                    and veh.s + veh.length / 2 < self.signal.loc):
                light_gap = self.signal.loc - (veh.s + veh.length / 2)
                if blocker_gap is None or light_gap < blocker_gap:
                    blocker_gap = light_gap
                    max_s = self.signal.loc - veh.length / 2

            integrate_gap(veh, dt, blocker_gap)
            if max_s is not None and veh.s > max_s:
                veh.s = max_s
                veh.vel = 0.0

        while self.queue and self.queue[0].s > self.exit_s:
            v = self.queue.pop(0)
            self.passing_times.append(t - v.born)

    def try_spawn(self, t, dt, rate):
        if random.random() >= rate * dt:
            return False
        if self.queue and self.queue[-1].s < self.entry_s + self.min_gap:
            return False
        self.queue.append(Vehicle(s=self.entry_s, vel=10.0, born=t))
        return True

    @property
    def positions(self):
        return [v.s for v in self.queue]


def run_lights(rate_h, rate_v, T=300.0, dt=0.05, period=20.0,
               entry_s=-95.0, exit_s=65.0, seed=0):
    random.seed(seed)
    sig_h = Signal(loc=LIGHT_STOP_LINE, period=period, offset=0.0)
    sig_v = Signal(loc=LIGHT_STOP_LINE, period=period, offset=period / 2)
    lane_h = Lane(signal=sig_h, entry_s=entry_s, exit_s=exit_s)
    lane_v = Lane(signal=sig_v, entry_s=entry_s, exit_s=exit_s)
    n_steps = int(T / dt)
    for i in range(n_steps):
        t = i * dt
        lane_h.try_spawn(t, dt, rate_h)
        lane_v.try_spawn(t, dt, rate_v)
        lane_h.step(t, dt)
        lane_v.step(t, dt)
    return lane_h.passing_times, lane_v.passing_times, T
```

A.2 Roundabout core

Listing 2: `code/round_core.py`: ring registry, state-machine vehicle, four-arm roundabout simulation.

```
import math
import numpy as np
from dataclasses import dataclass, field

TWOPI = 2.0 * math.pi
ARM_COLORS = ["#1f77b4", "#2ca02c", "#d62728", "#9467bd"]
```

```python


@dataclass
class Vehicle:
    car_id: int
    arm: int
    exit_arm: int
    born: float
    r: float
    theta: float
    vel: float = 0.0
    state: str = "approach"
    color: str = "#1f77b4"
    sigma: float = 7.0
    relax: float = 2.0
    car_length: float = 4.5
    entry_time: float = -1.0
    exit_ring_time: float = -1.0
    finish_time: float = -1.0


def lj_brake_acc(sigma, gap, vel):
    g = gap if gap > 0.5 else 0.5
    return -((sigma / g) ** 12) * vel


def relax_step(veh, dt, blocker_gap, vmax):
    drive = (vmax - veh.vel) / veh.relax
    brake = 0.0 if blocker_gap is None else lj_brake_acc(veh.sigma, blocker_gap, veh.vel)
    veh.vel = max(0.0, veh.vel + (drive + brake) * dt)


def arm_angle(arm):
    return (arm % 4) * (math.pi / 2)


def passed_target(old_theta, new_theta, target):
    travelled = (new_theta - old_theta) % TWOPI
    needed = (target - old_theta) % TWOPI
    return needed <= travelled and travelled > 1e-9


def _can_enter_ring(car, ring_cars, radius, min_gap, reaction_time, capacity_gap):
    if not ring_cars:
        return True
    max_capacity = max(1, int((TWOPI * radius) // capacity_gap))
    if len(ring_cars) >= max_capacity:
        return False
    entry = arm_angle(car.arm)
    speed_scale = max([car.vel] + [c.vel for c in ring_cars])
    yield_gap = max(min_gap, reaction_time * speed_scale)
    follow_gap = max(min_gap, car.car_length + 2.0)
    for other in ring_cars:
        ccw_dist = radius * ((entry - other.theta) % TWOPI)
        ahead_dist = radius * ((other.theta - entry) % TWOPI)
        if 0.0 < ccw_dist < yield_gap:
            return False
        if 0.0 < ahead_dist < follow_gap:
```

```
            return False
    return True


def _update_approach(cars, t, dt, radius, road_vmax, ring_vmax,
                     min_gap, reaction_time, stop_buffer, capacity_gap):
    stop_r = radius + stop_buffer
    ring_cars = [c for c in cars if c.state == "circle"]
    for arm in range(4):
        approaching = sorted(
            (c for c in cars if c.state == "approach" and c.arm == arm),
            key=lambda c: c.r,
        )
        for i, car in enumerate(approaching):
            if i == 0:
                allowed = _can_enter_ring(car, ring_cars, radius,
                                          min_gap, reaction_time, capacity_gap)
                blocked = car.r <= stop_r + 0.25 and not allowed
                gap = 0.1 if blocked else None
            else:
                front = approaching[i - 1]
                gap = car.r - front.r - car.car_length
            speed_limit = min(road_vmax,
                              ring_vmax * 1.4 if car.r < radius + 18.0 else road_vmax)
            relax_step(car, dt, gap, speed_limit)
            car.r -= car.vel * dt
            if car.r <= stop_r:
                if _can_enter_ring(car, ring_cars, radius,
                                   min_gap, reaction_time, capacity_gap):
                    car.state = "circle"
                    car.r = radius
                    car.theta = arm_angle(car.arm)
                    car.vel = max(0.6, min(car.vel, ring_vmax))
                    car.entry_time = t
                    ring_cars.append(car)
                else:
                    car.r = stop_r
                    car.vel = 0.0


def _update_circle(cars, t, dt, radius, ring_vmax, follow_gap_min):
    ring_cars = [c for c in cars if c.state == "circle"]
    if not ring_cars:
        return
    ring_cars.sort(key=lambda c: c.theta % TWOPI)
    n = len(ring_cars)
    for i, car in enumerate(ring_cars):
        if n == 1:
            gap = None
        else:
            front = ring_cars[(i + 1) % n]
            gap = max(0.0, radius * ((front.theta - car.theta) % TWOPI) - car.car_length)
        relax_step(car, dt, gap, ring_vmax)
        if gap is not None and gap < follow_gap_min:
            car.vel = max(0.0, car.vel + lj_brake_acc(car.sigma, max(gap, 0.5), car.vel) * dt)
    for car in list(ring_cars):
```

```
        old_theta = car.theta % TWOPI
        car.theta = (car.theta + car.vel * dt / radius) % TWOPI
        target = arm_angle(car.exit_arm)
        if passed_target(old_theta, car.theta, target):
            car.state = "exit"
            car.r = radius
            car.theta = target
            car.exit_ring_time = t


def _update_exits(cars, t, dt, road_vmax, finish_r):
    for exit_arm in range(4):
        exiting = sorted(
            (c for c in cars if c.state == "exit" and c.exit_arm == exit_arm),
            key=lambda c: -c.r,
        )
        for i, car in enumerate(exiting):
            gap = None if i == 0 else exiting[i - 1].r - car.r - car.car_length
            relax_step(car, dt, gap, road_vmax)
            car.r += car.vel * dt
            if car.r >= finish_r:
                car.state = "done"
                car.finish_time = t


def _can_spawn(cars, arm, spawn_r, min_spawn_gap):
    same = [c for c in cars if c.state == "approach" and c.arm == arm]
    if not same:
        return True
    return max(c.r for c in same) < spawn_r - min_spawn_gap


def run_roundabout_sim(rate_h=0.08, rate_v=0.08, opposite=False, spawn_rates=None,
                       T=220.0, dt=0.1, seed=0,
                       radius=15.0, road_length=95.0, exit_length=35.0,
                       road_vmax=11.0, ring_vmax=5.5,
                       min_gap=8.0, reaction_time=1.2,
                       follow_gap_min=7.5, capacity_gap=7.0,
                       min_spawn_gap=10.0,
                       record=False, record_stride=5,
                       warmup=50.0):
    rng = np.random.default_rng(seed)
    if spawn_rates is None:
        if opposite:
            spawn_rates = [rate_h, rate_v, rate_h, rate_v]
        else:
            spawn_rates = [0.0, 0.0, rate_h, rate_v]
    spawn_rates = np.asarray(spawn_rates, dtype=float)

    finish_r = radius + exit_length
    cars = []
    frames = []
    next_id = 0
    n_steps = int(math.ceil(T / dt))

    for step in range(n_steps):
        t = step * dt
        for arm, rate in enumerate(spawn_rates):
            arrivals = int(rng.poisson(rate * dt))
```

```
            for _ in range(arrivals):
                if _can_spawn(cars, arm, road_length, min_spawn_gap):
                    exit_arm = (arm + 2) % 4
                    cars.append(Vehicle(
                        car_id=next_id, arm=arm, exit_arm=exit_arm,
                        born=t, r=road_length, theta=arm_angle(arm),
                        vel=road_vmax * 0.65,
                        color=ARM_COLORS[arm],
                    ))
                    next_id += 1
        _update_approach(cars, t, dt, radius, road_vmax, ring_vmax,
                         min_gap, reaction_time, stop_buffer=5.0, capacity_gap=capacity_gap)
        _update_circle(cars, t, dt, radius, ring_vmax, follow_gap_min)
        _update_exits(cars, t, dt, road_vmax, finish_r)
        if record and step % record_stride == 0:
            frames.append(_snapshot(cars, t))

    return _summarize(cars, T, warmup, radius, road_length, finish_r, spawn_rates, frames)


def _snapshot(cars, t):
    items = []
    for c in cars:
        if c.state == "done":
            continue
        x = c.r * math.cos(c.theta)
        y = c.r * math.sin(c.theta)
        items.append({
            "x": x, "y": y, "theta": c.theta,
            "color": c.color, "arm": c.arm, "state": c.state,
        })
    return {"time": t, "cars": items}


def _summarize(cars, T, warmup, radius, road_length, finish_r, spawn_rates, frames):
    finished = [c for c in cars if c.finish_time >= 0]
    measured = [c for c in finished if c.finish_time >= warmup and c.born >= warmup]
    if len(measured) < 3:
        measured = finished
    times_h, times_v = [], []
    for c in measured:
        total = c.finish_time - c.born
        if c.arm in (0, 2):
            times_h.append(total)
        else:
            times_v.append(total)
    flow_meas_T = max(T - warmup, 1e-9)
    n_finished_warm = sum(1 for c in finished if c.finish_time >= warmup)
    return {
        "times_h": times_h,
        "times_v": times_v,
        "T": T,
        "warmup": warmup,
        "radius": radius,
        "road_length": road_length,
        "finish_r": finish_r,
        "spawn_rates": spawn_rates,
```

```
        "frames": frames,
        "n_cars": len(cars),
        "n_finished": len(finished),
        "n_measured": len(measured),
        "flow": n_finished_warm / flow_meas_T,
    }


def run_roundabout(rate_h, rate_v, T=220.0, dt=0.1, seed=0, opposite=False, **kw):
    out = run_roundabout_sim(rate_h=rate_h, rate_v=rate_v, T=T, dt=dt,
                             seed=seed, opposite=opposite, **kw)
    return out["times_h"], out["times_v"], out["T"]
```

A.3 Step demos

Listing 3: code/step1_two_cars.py

```
import matplotlib.pyplot as plt

from sim_core import Vehicle, integrate


def schedule(t, leader):
    leader.v_target = 0.0 if 10.0 <= t < 30.0 else 15.0


def main():
    leader = Vehicle(s=50.0, vel=15.0)
    follower = Vehicle(s=0.0, vel=15.0)

    dt = 0.01
    T = 60.0
    n = int(T / dt)

    times, s_l, s_f, v_l, v_f, gap = [], [], [], [], [], []
    for i in range(n):
        t = i * dt
        schedule(t, leader)
        integrate(leader, dt, blocker_s=None)
        integrate(follower, dt, blocker_s=leader.s)
        times.append(t)
        s_l.append(leader.s)
        s_f.append(follower.s)
        v_l.append(leader.vel)
        v_f.append(follower.vel)
        gap.append(leader.s - follower.s)

    fig, axes = plt.subplots(3, 1, figsize=(8, 8), sharex=True)
    axes[0].plot(times, s_l, color="firebrick", label="leader")
    axes[0].plot(times, s_f, color="steelblue", label="follower")
    axes[0].set_ylabel("arc length s (m)")
    axes[0].legend()

    axes[1].plot(times, v_l, color="firebrick", label="leader")
    axes[1].plot(times, v_f, color="steelblue", label="follower")
    axes[1].set_ylabel("velocity (m/s)")
    axes[1].legend()

```

```python
    axes[2].plot(times, gap, color="black")
    axes[2].axhline(7.0, ls="--", color="gray", label=r"$\sigma$")
    axes[2].set_xlabel("time (s)")
    axes[2].set_ylabel("gap (m)")
    axes[2].legend()

    fig.suptitle("Two-vehicle Lennard-Jones car-following test")
    plt.tight_layout()
    plt.savefig("../figures/fig_step1_two_vehicles.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 4: code/step2_pedestrian_light.py

```python
import matplotlib.pyplot as plt

from sim_core import Lane, Signal, Vehicle


def main():
    dt = 0.01
    T = 60.0
    n = int(T / dt)

    sig = Signal(loc=0.0, period=20.0, offset=0.0)
    lane = Lane(signal=sig, entry_s=-200.0, exit_s=300.0)

    N = 8
    for i in range(N):
        lane.queue.append(Vehicle(s=-200.0 + i * 20.0, vel=10.0, born=0.0))
    lane.queue.reverse()

    times, frames = [], []
    for i in range(n):
        t = i * dt
        lane.step(t, dt)
        if i % 5 == 0:
            times.append(t)
            frames.append([v.s for v in lane.queue])

    max_track = max(len(fr) for fr in frames)
    tracks = [[] for _ in range(max_track)]
    track_t = [[] for _ in range(max_track)]
    for fi, fr in enumerate(frames):
        for k, s in enumerate(fr):
            tracks[k].append(s)
            track_t[k].append(times[fi])

    fig, axes = plt.subplots(2, 1, figsize=(11, 6))
    for k in range(len(tracks)):
        axes[0].plot(track_t[k], tracks[k], lw=0.9)
    axes[0].axhline(sig.loc, color="red", ls="--", alpha=0.6, label="stop line")
    axes[0].set_xlabel("time (s)")
    axes[0].set_ylabel("position (m)")
    axes[0].set_title("Fleet trajectories through a pedestrian light (period 20 s)")
```

```
    axes[0].legend()
    axes[0].grid(alpha=0.3)

    snap = frames[-1]
    axes[1].set_xlim(-220, 320)
    axes[1].set_ylim(-1, 1)
    axes[1].set_yticks([])
    axes[1].axhline(0, color="lightgray", lw=8, zorder=0)
    axes[1].plot(snap, [0] * len(snap), marker="s", linestyle="",
                 markersize=12, color="midnightblue")
    is_red = sig.red(times[-1])
    axes[1].plot([sig.loc], [0], marker="o", markersize=18,
                 color=("crimson" if is_red else "limegreen"))
    axes[1].set_xlabel("position (m)")
    axes[1].set_title(f"Snapshot at t = {times[-1]:.1f} s, light = {'red' if is_red
    else 'green'}")

    plt.tight_layout()
    plt.savefig("../figures/fig_step2_pedestrian_light.png", dpi=150, bbox_inches="
    tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 5: code/step3_intersection_lights.py

```
import math
import random
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Rectangle

from sim_core import Lane, LIGHT_STOP_LINE, Signal


GRASS = "#9bc78a"
ASPHALT = "#3b3b3b"
EDGE = "#d9d9d9"
CENTER = "#f1c40f"
STOP_LINE = "#ffffff"


def draw_flow_arrows(ax):
    arrow = dict(arrowstyle="-|>", color="white", lw=2.5,
                 mutation_scale=15)
    label_box = dict(facecolor="black", edgecolor="none", alpha=0.45,
                     boxstyle="round,pad=0.2")
    ax.annotate("", xy=(-150, -34), xytext=(-205, -34),
                arrowprops=arrow, zorder=7)
    ax.text(-178, -50, "H flow", color="white", fontsize=11,
            ha="center", va="center", bbox=label_box, zorder=7)
    ax.annotate("", xy=(34, -150), xytext=(34, -205),
                arrowprops=arrow, zorder=7)
    ax.text(52, -178, "V flow", color="white", fontsize=11,
            ha="center", va="center", rotation=90, bbox=label_box, zorder=7)


```

```python
def draw_intersection(ax, road_w=24.0, span=230.0, stop_s=LIGHT_STOP_LINE):
    ax.add_patch(Rectangle((-span, -span), 2 * span, 2 * span,
                           facecolor=GRASS, edgecolor="none", zorder=0))
    ax.add_patch(Rectangle((-span, -road_w / 2), 2 * span, road_w,
                           facecolor=ASPHALT, edgecolor="none", zorder=1))
    ax.add_patch(Rectangle((-road_w / 2, -span), road_w, 2 * span,
                           facecolor=ASPHALT, edgecolor="none", zorder=1))
    ax.plot([-span, span], [road_w / 2, road_w / 2], color=EDGE, lw=1.0, zorder=2)
    ax.plot([-span, span], [-road_w / 2, -road_w / 2], color=EDGE, lw=1.0, zorder=2)
    ax.plot([road_w / 2, road_w / 2], [-span, span], color=EDGE, lw=1.0, zorder=2)
    ax.plot([-road_w / 2, -road_w / 2], [-span, span], color=EDGE, lw=1.0, zorder=2)
    for x0 in np.arange(-span, span, 16):
        if abs(x0 + 8) > road_w / 2:
            ax.plot([x0, x0 + 8], [0, 0], color=CENTER, lw=1.5, zorder=2)
    for y0 in np.arange(-span, span, 16):
        if abs(y0 + 8) > road_w / 2:
            ax.plot([0, 0], [y0, y0 + 8], color=CENTER, lw=1.5, zorder=2)
    ax.plot([-road_w / 2, road_w / 2], [-road_w / 2, -road_w / 2],
            color=STOP_LINE, lw=3, zorder=3)
    ax.plot([-road_w / 2, road_w / 2], [road_w / 2, road_w / 2],
            color=STOP_LINE, lw=3, zorder=3)
    ax.plot([-road_w / 2, -road_w / 2], [-road_w / 2, road_w / 2],
            color=STOP_LINE, lw=3, zorder=3)
    ax.plot([road_w / 2, road_w / 2], [-road_w / 2, road_w / 2],
            color=STOP_LINE, lw=3, zorder=3)
    half = road_w / 2
    ax.plot([stop_s, stop_s], [-half, 0],
            color=STOP_LINE, lw=3.5, zorder=3)
    ax.plot([-stop_s, -stop_s], [0, half],
            color=STOP_LINE, lw=3.5, zorder=3)
    ax.plot([0, half], [stop_s, stop_s],
            color=STOP_LINE, lw=3.5, zorder=3)
    ax.plot([-half, 0], [-stop_s, -stop_s],
            color=STOP_LINE, lw=3.5, zorder=3)
    draw_flow_arrows(ax)


def car_rect(ax, x, y, orientation, color, w=8, l=14):
    if orientation == "h":
        ax.add_patch(Rectangle((x - l / 2, y - w / 2), l, w,
                               facecolor=color, edgecolor="black", lw=0.8, zorder=4))
    else:
        ax.add_patch(Rectangle((x - w / 2, y - l / 2), w, l,
                               facecolor=color, edgecolor="black", lw=0.8, zorder=4))


def draw_signal(ax, h_red, v_red):
    ax.add_patch(Rectangle((-22, -22), 44, 44, facecolor="#222",
                           edgecolor="black", lw=1.2, zorder=5))
    ax.add_patch(plt.Circle((-10, 0), 6,
                            facecolor=("#e74c3c" if h_red else "#2ecc71"),
                            edgecolor="black", lw=0.8, zorder=6))
    ax.add_patch(plt.Circle((10, 0), 6,
                            facecolor=("#e74c3c" if v_red else "#2ecc71"),
                            edgecolor="black", lw=0.8, zorder=6))
    ax.text(-10, -16, "H", color="white", ha="center", fontsize=8, zorder=6)
    ax.text(10, -16, "V", color="white", ha="center", fontsize=8, zorder=6)
```

```python


def main():
    random.seed(7)
    dt = 0.01
    T = 90.0
    period = 20.0
    n = int(T / dt)
    lane_off = 6.0

    sig_h = Signal(loc=LIGHT_STOP_LINE, period=period, offset=0.0)
    sig_v = Signal(loc=LIGHT_STOP_LINE, period=period, offset=period / 2)
    lane_e = Lane(signal=sig_h, entry_s=-200.0, exit_s=200.0)
    lane_w = Lane(signal=sig_h, entry_s=-200.0, exit_s=200.0)
    lane_n = Lane(signal=sig_v, entry_s=-200.0, exit_s=200.0)
    lane_s = Lane(signal=sig_v, entry_s=-200.0, exit_s=200.0)

    snaps_e, snaps_w, snaps_n, snaps_s, ts = [], [], [], [], []
    for i in range(n):
        t = i * dt
        lane_e.try_spawn(t, dt, rate=0.4)
        lane_w.try_spawn(t, dt, rate=0.4)
        lane_n.try_spawn(t, dt, rate=0.4)
        lane_s.try_spawn(t, dt, rate=0.4)
        for ln in (lane_e, lane_w, lane_n, lane_s):
            ln.step(t, dt)
        if i % 30 == 0:
            ts.append(t)
            snaps_e.append(lane_e.positions)
            snaps_w.append(lane_w.positions)
            snaps_n.append(lane_n.positions)
            snaps_s.append(lane_s.positions)

    fig, ax = plt.subplots(figsize=(8.5, 8.5))
    ax.set_xlim(-230, 230)
    ax.set_ylim(-230, 230)
    ax.set_aspect("equal")
    ax.set_xticks([])
    ax.set_yticks([])

    draw_intersection(ax)

    snap_idx = -1
    t_show = ts[snap_idx]
    for s in snaps_e[snap_idx]:
        car_rect(ax, s, -lane_off, "h", "#1f3a93")
    for s in snaps_w[snap_idx]:
        car_rect(ax, -s, lane_off, "h", "#2e86c1")
    for s in snaps_n[snap_idx]:
        car_rect(ax, lane_off, s, "v", "#e67e22")
    for s in snaps_s[snap_idx]:
        car_rect(ax, -lane_off, -s, "v", "#d35400")

    h_red = sig_h.red(t_show)
    v_red = sig_v.red(t_show)
    draw_signal(ax, h_red, v_red)

    ax.set_title(f"Crossroads with opposing-phase traffic lights "
                 f"(t = {t_show:.0f} s, H={'R' if h_red else 'G'}, V={'R' if v_red
```

```
    else 'G'})",
                  fontsize=12)
    plt.tight_layout()
    plt.savefig("../figures/fig_step3_intersection_lights.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 6: code/step4_roundabout_demo.py

```
import math
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.patches import Circle, Rectangle

from round_core import run_roundabout_sim, ARM_COLORS


def draw_base_roads(ax, radius=15.0, extent=105.0, road_width=14.0):
    ax.add_patch(Rectangle((-extent, -extent), 2 * extent, 2 * extent,
                           facecolor="#cfe7c4", edgecolor="none", zorder=0))
    ax.add_patch(Rectangle((-extent, -road_width / 2), 2 * extent, road_width,
                           facecolor="#a9a9a9", edgecolor="none", zorder=1))
    ax.add_patch(Rectangle((-road_width / 2, -extent), road_width, 2 * extent,
                           facecolor="#a9a9a9", edgecolor="none", zorder=1))

    ring_half = road_width / 4.0
    gap = radius + ring_half
    for sgn in (-1, 1):
        ax.plot([sgn * extent, sgn * gap], [0, 0],
                ls="--", color="#444", lw=1.4, dashes=(6, 6), zorder=2)
        ax.plot([0, 0], [sgn * extent, sgn * gap],
                ls="--", color="#444", lw=1.4, dashes=(6, 6), zorder=2)

    ax.add_patch(Circle((0, 0), radius + ring_half,
                        facecolor="#7d7d7d", edgecolor="none", zorder=3))
    ax.add_patch(Circle((0, 0), max(radius - ring_half, 0.5),
                        facecolor="#5d8a48", edgecolor="black", lw=1.0, zorder=4))
    theta = np.linspace(0, 2 * math.pi, 200)
    ax.plot(radius * np.cos(theta), radius * np.sin(theta),
            color="white", lw=1.0, zorder=5)


def main():
    out = run_roundabout_sim(
        rate_h=0.10, rate_v=0.10, opposite=True,
        T=140.0, dt=0.1, seed=3,
        record=True, record_stride=5,
    )
    radius = out["radius"]
    extent = out["road_length"] + 8.0

    snap = out["frames"][-1]
    t_show = snap["time"]

    fig, ax = plt.subplots(figsize=(8.5, 8.5))
```

```python
    ax.set_aspect("equal")
    ax.set_xlim(-extent, extent)
    ax.set_ylim(-extent, extent)
    ax.set_xticks([])
    ax.set_yticks([])

    draw_base_roads(ax, radius=radius, extent=extent)

    lane_offset = 3.0
    if snap["cars"]:
        xs, ys, cs = [], [], []
        for c in snap["cars"]:
            x, y = c["x"], c["y"]
            theta = c.get("theta")
            if theta is None:
                theta = math.atan2(y, x)
            nx, ny = -math.sin(theta), math.cos(theta)
            if c["state"] == "approach":
                x += lane_offset * nx
                y += lane_offset * ny
            elif c["state"] == "exit":
                x -= lane_offset * nx
                y -= lane_offset * ny
            xs.append(x); ys.append(y); cs.append(c["color"])
        ax.scatter(xs, ys, s=70, c=cs, edgecolor="black", lw=0.6, zorder=10)

    handles = []
    for arm in range(4):
        h = ax.scatter([], [], s=70, c=ARM_COLORS[arm], edgecolor="black", lw=0.6,
                       label=f"arm {arm} ({['E', 'N', 'W', 'S'][arm]})")
        handles.append(h)
    ax.legend(handles=handles, loc="upper right", fontsize=9, framealpha=0.85)

    ax.set_title(f"Four-arm roundabout, all four arms emitting (t = {t_show:.0f} s)"
    ,
                 fontsize=12)
    plt.tight_layout()
    plt.savefig("../figures/fig_step4_roundabout.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

A.4 Analyses

Listing 7: code/analysis1_roundabout_baseline.py

```python
import matplotlib.pyplot as plt
import numpy as np

from round_core import run_roundabout


def main():
    rate = 0.3
    times_h, times_v, T = run_roundabout(rate_h=rate, rate_v=rate, T=300.0,
                                         dt=0.05, seed=11)
```

```python

    mh = float(np.mean(times_h)) if times_h else float("nan")
    mv = float(np.mean(times_v)) if times_v else float("nan")
    all_t = times_h + times_v
    m_all = float(np.mean(all_t)) if all_t else float("nan")

    print(f"Horizontal: N={len(times_h):4d}  mean passing time = {mh:6.2f} s")
    print(f"Vertical:   N={len(times_v):4d}  mean passing time = {mv:6.2f} s")
    print(f"Combined:   N={len(all_t):4d}  mean passing time = {m_all:6.2f} s")

    fig, ax = plt.subplots(figsize=(8, 4.5))
    bins = np.linspace(0, max(all_t) * 1.05 if all_t else 100, 30)
    ax.hist(times_h, bins=bins, color="navy", alpha=0.55,
            label=f"horizontal  (mean = {mh:.1f} s)")
    ax.hist(times_v, bins=bins, color="darkorange", alpha=0.55,
            label=f"vertical    (mean = {mv:.1f} s)")
    ax.axvline(mh, color="navy", ls="--", lw=1)
    ax.axvline(mv, color="darkorange", ls="--", lw=1)
    ax.set_xlabel("passing time (s)")
    ax.set_ylabel("number of cars")
    ax.set_title(f"Roundabout passing-time distribution "
                 f"(spawn = {rate} cars/s/road, T = {T:.0f} s)")
    ax.legend()
    ax.grid(alpha=0.25)
    plt.tight_layout()
    plt.savefig("../figures/fig_analysis1_roundabout_baseline.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 8: code/analysis2_roundabout_density.py

```python
import matplotlib.pyplot as plt
import numpy as np

from round_core import run_roundabout


def main():
    rates = np.array([0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.80])
    means, throughputs = [], []
    print(f"{'rate':>6}  {'<t>':>7}  {'flow':>7}  {'N':>5}")
    for r in rates:
        th, tv, T = run_roundabout(rate_h=r, rate_v=r, T=200.0, dt=0.05, seed=21)
        all_t = th + tv
        m = float(np.mean(all_t)) if all_t else float("nan")
        flow = len(all_t) / T
        means.append(m)
        throughputs.append(flow)
        print(f"{r:6.2f}  {m:7.2f}  {flow:7.3f}  {len(all_t):5d}")

    fig, ax = plt.subplots(1, 2, figsize=(12, 4.2))
    ax[0].plot(rates, means, marker="D", color="firebrick")
    ax[0].set_xlabel("spawn rate per road (cars / s)")
    ax[0].set_ylabel("mean passing time (s)")
    ax[0].set_title("Roundabout: passing time vs density")
```

```python
    ax[0].grid(alpha=0.3)

    ax[1].plot(rates, throughputs, marker="D", color="firebrick", label="exit flow")
    ax[1].plot(rates, 2 * rates, color="gray", ls="--", label="2 × spawn (no losses)")
    ax[1].set_xlabel("spawn rate per road (cars / s)")
    ax[1].set_ylabel("total exit flow (cars / s)")
    ax[1].set_title("Roundabout: throughput")
    ax[1].grid(alpha=0.3)
    ax[1].legend()

    plt.tight_layout()
    plt.savefig("../figures/fig_analysis2_roundabout_density.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 9: code/analysis3_roundabout_asymmetric.py

```python
import matplotlib.pyplot as plt
import numpy as np

from round_core import run_roundabout


def main():
    rates_h = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    rates_v = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    grid = np.zeros((len(rates_h), len(rates_v)))

    print(f"Asymmetric sweep: {len(rates_h)} x {len(rates_v)} grid")
    for i, rh in enumerate(rates_h):
        for j, rv in enumerate(rates_v):
            th, tv, T = run_roundabout(rate_h=rh, rate_v=rv, T=150.0, dt=0.05, seed=31)
            all_t = th + tv
            grid[i, j] = float(np.mean(all_t)) if all_t else float("nan")
            print(f"  rate_h={rh:.2f}  rate_v={rv:.2f}  <t>={grid[i,j]:6.2f} s  N={len(all_t)}")

    fig, ax = plt.subplots(figsize=(7.5, 6))
    extent = [rates_v.min() - 0.05, rates_v.max() + 0.05,
              rates_h.min() - 0.05, rates_h.max() + 0.05]
    im = ax.imshow(grid, origin="lower", extent=extent, aspect="auto",
                   cmap="magma", interpolation="bilinear")
    cs = ax.contour(rates_v, rates_h, grid, levels=8, colors="white", linewidths=0.7)
    ax.clabel(cs, inline=True, fontsize=8, fmt="%.0f")
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label("mean passing time (s)")
    ax.set_xlabel("vertical spawn rate (cars / s)")
    ax.set_ylabel("horizontal spawn rate (cars / s)")
    ax.set_title("Roundabout: mean passing time vs asymmetric spawn rates")
    ax.plot(rates_v, rates_h, color="cyan", ls=":", label="symmetric line")
    ax.legend()
    plt.tight_layout()
```

```python
    plt.savefig("../figures/fig_analysis3_roundabout_asymmetric.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 10: code/analysis4_roundabout_hv_difference.py

```python
import matplotlib.pyplot as plt
import numpy as np

from round_core import run_roundabout


def main():
    rates_h = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    rates_v = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    diff = np.zeros((len(rates_h), len(rates_v)))

    print(f"H-V difference sweep: {len(rates_h)} x {len(rates_v)} grid")
    for i, rh in enumerate(rates_h):
        for j, rv in enumerate(rates_v):
            th, tv, T = run_roundabout(rate_h=rh, rate_v=rv, T=150.0, dt=0.05, seed=41)
            mh = float(np.mean(th)) if th else float("nan")
            mv = float(np.mean(tv)) if tv else float("nan")
            diff[i, j] = mh - mv
            print(f"  rate_h={rh:.2f}  rate_v={rv:.2f}  H={mh:6.2f}  V={mv:6.2f}  H-V={diff[i,j]:+6.2f}")

    fig, ax = plt.subplots(figsize=(7.5, 6))
    vmax = float(np.nanmax(np.abs(diff)))
    extent = [rates_v.min() - 0.05, rates_v.max() + 0.05,
              rates_h.min() - 0.05, rates_h.max() + 0.05]
    im = ax.imshow(diff, origin="lower", extent=extent, aspect="auto",
                   cmap="seismic", vmin=-vmax, vmax=vmax, interpolation="bilinear")
    cs = ax.contour(rates_v, rates_h, diff, levels=[0], colors="black", linewidths=1.5)
    ax.clabel(cs, inline=True, fontsize=9, fmt="%.0f")
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label(r"$\langle t \rangle_H - \langle t \rangle_V$  (s)")
    ax.set_xlabel("vertical spawn rate (cars / s)")
    ax.set_ylabel("horizontal spawn rate (cars / s)")
    ax.set_title("Roundabout: H - V passing-time bias")
    ax.plot(rates_v, rates_h, color="black", ls=":", label="symmetric line")
    ax.legend()
    plt.tight_layout()
    plt.savefig("../figures/fig_analysis4_roundabout_hv_difference.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 11: code/analysis5_lights_density_and_cycle.py

```python
import matplotlib.pyplot as plt
import numpy as np

from sim_core import run_lights


def main():
    rates = np.array([0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.80])
    means_density = []
    print(f"Density sweep (period = 20 s):")
    print(f"{'rate':>6}  {'<t>':>7}  {'N':>5}")
    for r in rates:
        th, tv, T = run_lights(rate_h=r, rate_v=r, T=300.0, period=20.0, seed=51)
        all_t = th + tv
        m = float(np.mean(all_t)) if all_t else float("nan")
        means_density.append(m)
        print(f"{r:6.2f}  {m:7.2f}  {len(all_t):5d}")

    cycles = np.array([5, 10, 15, 20, 25, 30, 40, 50, 60, 80], dtype=float)
    means_cycle = []
    fixed_rate = 0.4
    print(f"\nCycle sweep (rate = {fixed_rate} cars/s/road):")
    print(f"{'cycle':>6}  {'<t>':>7}  {'N':>5}")
    for c in cycles:
        th, tv, T = run_lights(rate_h=fixed_rate, rate_v=fixed_rate, T=300.0,
                               period=c, seed=52)
        all_t = th + tv
        m = float(np.mean(all_t)) if all_t else float("nan")
        means_cycle.append(m)
        print(f"{c:6.1f}  {m:7.2f}  {len(all_t):5d}")

    fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
    ax[0].plot(rates, means_density, marker="o", color="forestgreen")
    ax[0].set_xlabel("spawn rate per road (cars / s)")
    ax[0].set_ylabel("mean passing time (s)")
    ax[0].set_title("Traffic light: passing time vs density (20 s cycle)")
    ax[0].grid(alpha=0.3)

    ax[1].plot(cycles, means_cycle, marker="^", color="forestgreen")
    ax[1].set_xlabel("cycle period (s)")
    ax[1].set_ylabel("mean passing time (s)")
    ax[1].set_title(f"Traffic light: passing time vs cycle (rate = {fixed_rate})")
    ax[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig("../figures/fig_analysis5_lights_density_and_cycle.png", dpi=150,
    bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 12: code/analysis6_roundabout_vs_lights.py

```python
import matplotlib.pyplot as plt
import numpy as np

```

```python
from round_core import run_roundabout
from sim_core import run_lights


def main():
    rates = np.array([0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.80])
    means_round = []
    means_lights = []

    print(f"{'rate':>6}  {'<t>_round':>10}  {'<t>_lights':>11}")
    for r in rates:
        th_r, tv_r, T = run_roundabout(rate_h=r, rate_v=r, T=200.0, dt=0.05, seed=61)
        th_l, tv_l, _ = run_lights(rate_h=r, rate_v=r, T=200.0, period=20.0, seed=62)
        m_r = float(np.mean(th_r + tv_r)) if (th_r or tv_r) else float("nan")
        m_l = float(np.mean(th_l + tv_l)) if (th_l or tv_l) else float("nan")
        means_round.append(m_r)
        means_lights.append(m_l)
        print(f"{r:6.2f}  {m_r:10.2f}  {m_l:11.2f}")

    means_round = np.array(means_round)
    means_lights = np.array(means_lights)

    fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))
    ax[0].plot(rates, means_round, marker="D", color="firebrick", label="roundabout")
    ax[0].plot(rates, means_lights, marker="o", color="forestgreen", label="traffic light")
    ax[0].set_xlabel("spawn rate per road (cars / s)")
    ax[0].set_ylabel("mean passing time (s)")
    ax[0].set_title("Mean passing time: roundabout vs traffic light")
    ax[0].legend()
    ax[0].grid(alpha=0.3)

    diff = means_lights - means_round
    ax[1].bar(rates, diff, width=0.04,
              color=np.where(diff > 0, "firebrick", "forestgreen"),
              edgecolor="black", alpha=0.8)
    ax[1].axhline(0, color="black", lw=0.8)
    ax[1].set_xlabel("spawn rate per road (cars / s)")
    ax[1].set_ylabel(r"$\langle t \rangle_{lights} - \langle t \rangle_{round}$  (s)")
    ax[1].set_title("Where each control wins (positive = roundabout faster)")
    ax[1].grid(alpha=0.3, axis="y")

    plt.tight_layout()
    plt.savefig("../figures/fig_analysis6_roundabout_vs_lights.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 13: code/analysis7_lights_asymmetric.py

```python
import matplotlib.pyplot as plt
import numpy as np
```

```python

from sim_core import run_lights


def main():
    rates_h = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    rates_v = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    grid = np.zeros((len(rates_h), len(rates_v)))

    print(f"Lights asymmetric sweep: {len(rates_h)} x {len(rates_v)} grid")
    for i, rh in enumerate(rates_h):
        for j, rv in enumerate(rates_v):
            th, tv, T = run_lights(rate_h=rh, rate_v=rv, T=200.0, period=20.0, seed=71)
            all_t = th + tv
            grid[i, j] = float(np.mean(all_t)) if all_t else float("nan")
            print(f"  rate_h={rh:.2f}  rate_v={rv:.2f}  <t>={grid[i,j]:6.2f} s  N={len(all_t)}")

    fig, ax = plt.subplots(figsize=(7.5, 6))
    extent = [rates_v.min() - 0.05, rates_v.max() + 0.05,
              rates_h.min() - 0.05, rates_h.max() + 0.05]
    im = ax.imshow(grid, origin="lower", extent=extent, aspect="auto",
                   cmap="viridis", interpolation="bilinear")
    cs = ax.contour(rates_v, rates_h, grid, levels=8, colors="white", linewidths=0.7)
    ax.clabel(cs, inline=True, fontsize=8, fmt="%.0f")
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label("mean passing time (s)")
    ax.set_xlabel("vertical spawn rate (cars / s)")
    ax.set_ylabel("horizontal spawn rate (cars / s)")
    ax.set_title("Traffic light: mean passing time vs asymmetric spawn rates")
    ax.plot(rates_v, rates_h, color="cyan", ls=":", label="symmetric line")
    ax.legend()
    plt.tight_layout()
    plt.savefig("../figures/fig_analysis7_lights_asymmetric.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 14: code/analysis8_lights_hv_difference.py

```python
import matplotlib.pyplot as plt
import numpy as np

from sim_core import run_lights


def main():
    rates_h = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    rates_v = np.array([0.10, 0.20, 0.30, 0.40, 0.50, 0.60])
    diff = np.zeros((len(rates_h), len(rates_v)))

    print(f"Lights H-V difference sweep: {len(rates_h)} x {len(rates_v)} grid")
    for i, rh in enumerate(rates_h):
        for j, rv in enumerate(rates_v):
```

```python
            th, tv, T = run_lights(rate_h=rh, rate_v=rv, T=200.0, period=20.0, seed=81)
            mh = float(np.mean(th)) if th else float("nan")
            mv = float(np.mean(tv)) if tv else float("nan")
            diff[i, j] = mh - mv
            print(f"  rate_h={rh:.2f}  rate_v={rv:.2f}  H={mh:6.2f}  V={mv:6.2f}  H-V={diff[i,j]:+6.2f}")

    fig, ax = plt.subplots(figsize=(7.5, 6))
    vmax = float(np.nanmax(np.abs(diff)))
    if vmax < 1e-3:
        vmax = 1.0
    extent = [rates_v.min() - 0.05, rates_v.max() + 0.05,
              rates_h.min() - 0.05, rates_h.max() + 0.05]
    im = ax.imshow(diff, origin="lower", extent=extent, aspect="auto",
                   cmap="seismic", vmin=-vmax, vmax=vmax, interpolation="bilinear")
    cs = ax.contour(rates_v, rates_h, diff, levels=[0], colors="black", linewidths=1.5)
    ax.clabel(cs, inline=True, fontsize=9, fmt="%.0f")
    cbar = fig.colorbar(im, ax=ax)
    cbar.set_label(r"$\langle t \rangle_H - \langle t \rangle_V$  (s)")
    ax.set_xlabel("vertical spawn rate (cars / s)")
    ax.set_ylabel("horizontal spawn rate (cars / s)")
    ax.set_title("Traffic light: H - V passing-time difference")
    ax.plot(rates_v, rates_h, color="black", ls=":", label="symmetric line")
    ax.legend()
    plt.tight_layout()
    plt.savefig("../figures/fig_analysis8_lights_hv_difference.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

Listing 15: code/analysis9_flow_rate_comparison.py

```python
import matplotlib.pyplot as plt
import numpy as np

from round_core import run_roundabout
from sim_core import run_lights


def main():
    rates = np.array([0.05, 0.10, 0.20, 0.30, 0.40, 0.50, 0.60, 0.80])
    flow_round = []
    flow_lights = []

    print(f"{'rate':>6}  {'flow_round':>10}  {'flow_lights':>11}")
    for r in rates:
        th_r, tv_r, T = run_roundabout(rate_h=r, rate_v=r, T=200.0, dt=0.05, seed=91)
        th_l, tv_l, _ = run_lights(rate_h=r, rate_v=r, T=200.0, period=20.0, seed=92)
        fr = (len(th_r) + len(tv_r)) / T
        fl = (len(th_l) + len(tv_l)) / T
        flow_round.append(fr)
        flow_lights.append(fl)
```

```python
        print(f"{r:6.2f}  {fr:10.3f}  {fl:11.3f}")

    flow_round = np.array(flow_round)
    flow_lights = np.array(flow_lights)

    fig, ax = plt.subplots(1, 2, figsize=(13, 4.5))

    ax[0].plot(rates, flow_round, marker="D", color="firebrick", label="roundabout")
    ax[0].plot(rates, flow_lights, marker="o", color="forestgreen", label="traffic light")
    ax[0].plot(rates, 2 * rates, color="gray", ls="--", alpha=0.6,
               label="2 × spawn (no losses)")
    ax[0].set_xlabel("spawn rate per road (cars / s)")
    ax[0].set_ylabel("total exit flow (cars / s)")
    ax[0].set_title("Throughput: roundabout vs traffic light")
    ax[0].legend()
    ax[0].grid(alpha=0.3)

    saturation = np.where(2 * rates > 0, flow_round / (2 * rates), 0)
    sat_l = np.where(2 * rates > 0, flow_lights / (2 * rates), 0)
    ax[1].plot(rates, saturation, marker="D", color="firebrick", label="roundabout")
    ax[1].plot(rates, sat_l, marker="o", color="forestgreen", label="traffic light")
    ax[1].axhline(1.0, color="gray", ls="--", alpha=0.6, label="100 % efficiency")
    ax[1].set_xlabel("spawn rate per road (cars / s)")
    ax[1].set_ylabel("exit flow / spawn flow")
    ax[1].set_title("Throughput efficiency")
    ax[1].set_ylim(0, 1.15)
    ax[1].legend()
    ax[1].grid(alpha=0.3)

    plt.tight_layout()
    plt.savefig("../figures/fig_analysis9_flow_rate_comparison.png", dpi=150, bbox_inches="tight")
    plt.show()


if __name__ == "__main__":
    main()
```

B Supplementary figures

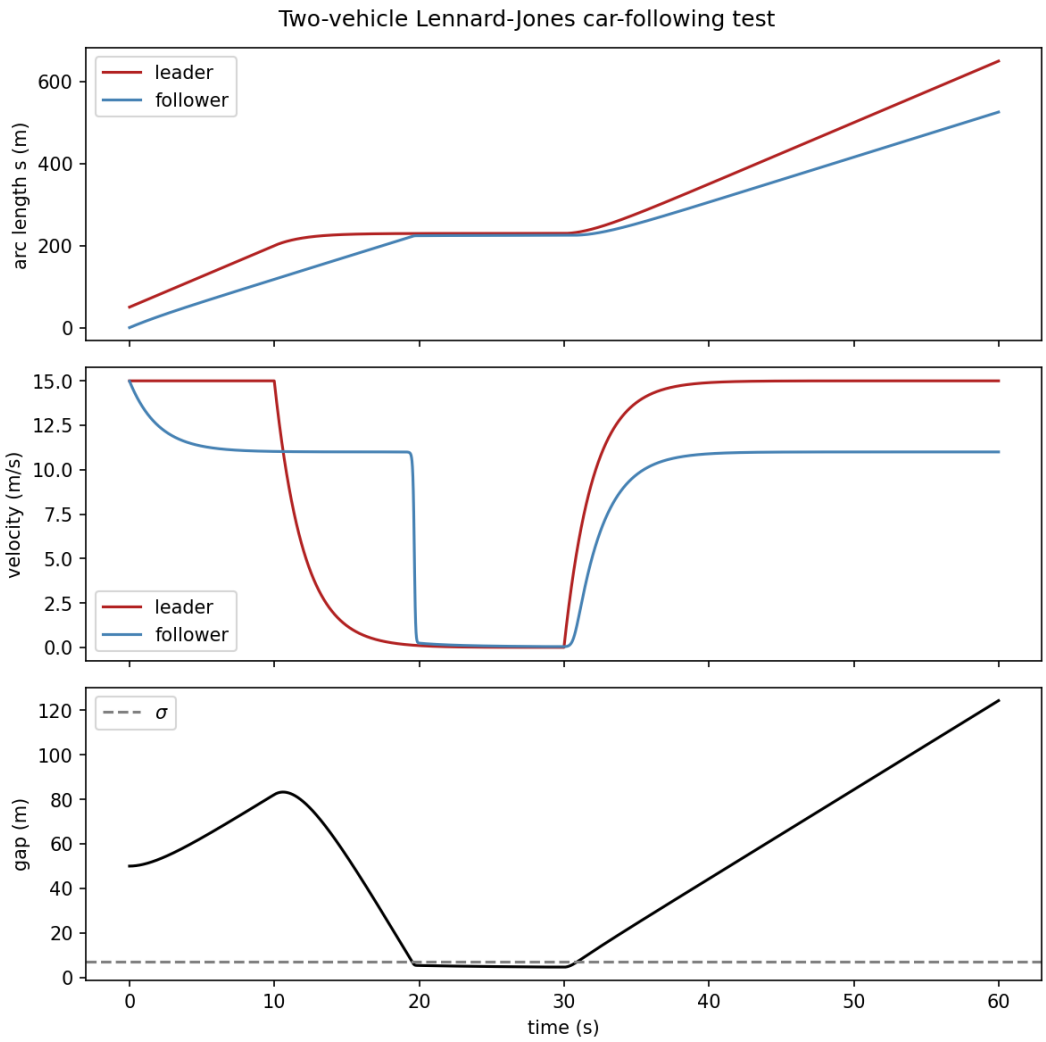


Figure 7: Two-vehicle stop-and-go test for the Lennard-Jones brake. Top: position. Middle: velocity. Bottom: gap, with the dashed line indicating σ. The follower brakes hard near σ and never collides with the leader.

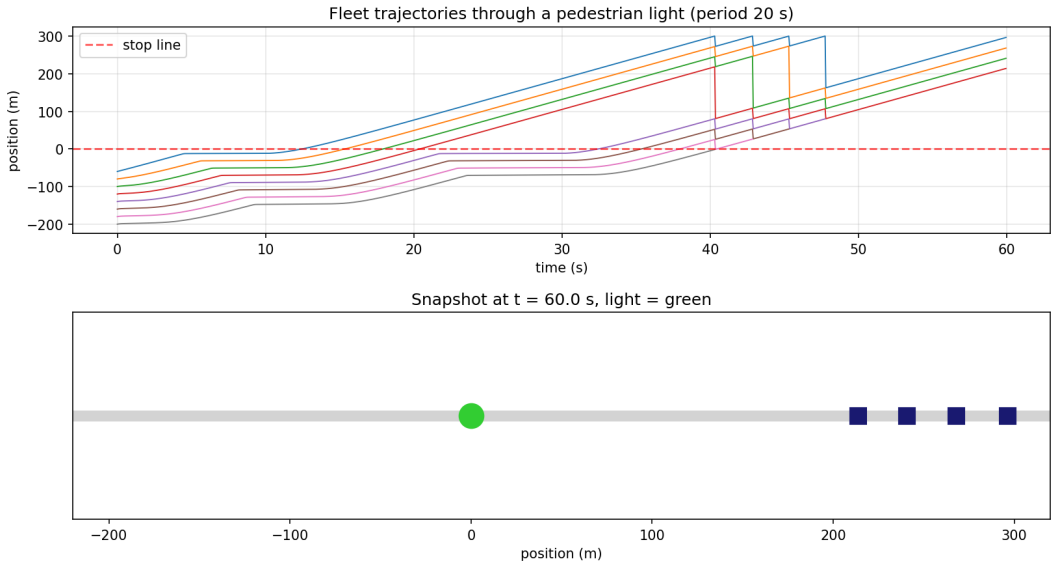


Figure 8: Eight-car fleet meeting a single pedestrian-crossing light with 20 s cycle. The fleet compresses against the stop line during red and releases as a wave once the light turns green.

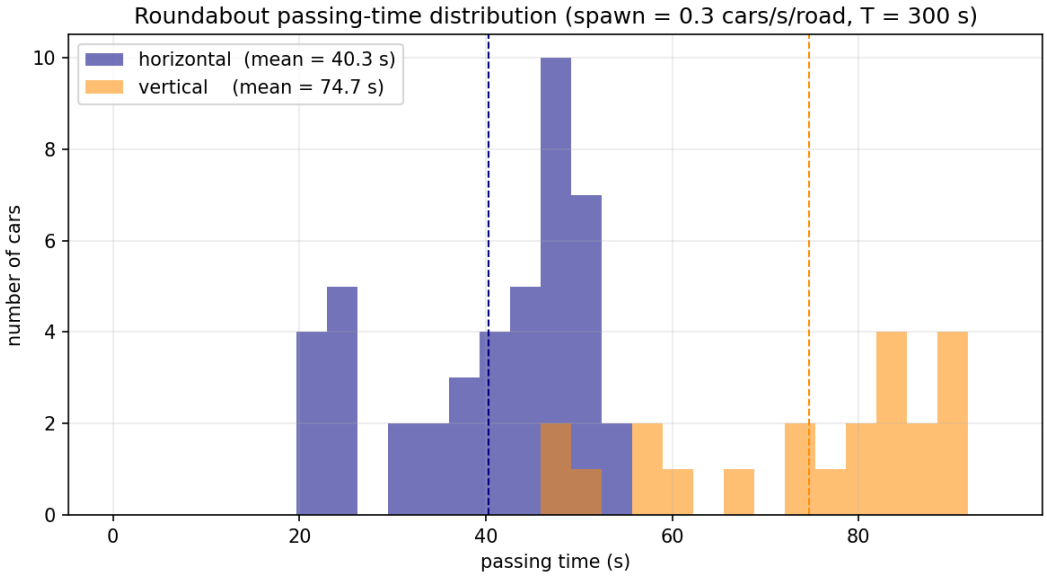


Figure 9: Baseline roundabout passing-time distribution at $r_H = r_V = 0.3$ cars/s. Horizontal cars cluster tightly near ~ 40 s; vertical cars sprawl out past 80 s with mean ~ 75 s.